

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

CAMPUS CORNÉLIO PROCÓPIO

Programação Orientada a Objetos 2

LUCAS BRUCK

MARIA JULIA PEDROSO

MARCELO YUJI RUIZ

WAGACLOUD

CORNÉLIO PROCÓPIO

JUNHO, 2025

Lucas Bruck

Maria Julia Pedroso

Marcelo Yuji Ruiz

WAGACLOUD

Projeto elaborado na disciplina de Programação Orientada a Objetos 2 do curso de Engenharia da Computação, do Campus Cornélio Procópio da Universidade Tecnológica Federal do Paraná.

Professora: Gisele Alves Santana

CORNÉLIO PROCÓPIO

JUNHO, 2025

RESUMO

Este trabalho apresenta o WagaCloud, um simulador de plataforma de cloud computing

desenvolvido para aplicar, na prática, os conceitos de Programação Orientada a Objetos.

O objetivo foi construir um sistema em arquitetura de camadas capaz de autenticar usuários, gerenciar recursos de nuvem — máquinas virtuais (VMs) e discos de armazenamento — e monitorar o uso desses recursos por meio de métricas simuladas de CPU e RAM. A metodologia adotou Java 21 com o framework Spring Boot, persistência em banco relacional PostgreSQL via Spring Data JPA (Hibernate) e um front-end leve em HTML, CSS e JavaScript consumindo uma API REST. O domínio foi modelado a partir de uma classe abstrata Recurso, da qual herdam VirtualMachine e Armazenamento, explorando herança, polimorfismo e encapsulamento. Como principais resultados, obteve-se um sistema funcional com dois níveis de acesso (ADMIN e COMUM), controle de ciclo de vida das VMs, anexação/expansão de discos e listagem de recursos em estado crítico. Conclui-se que o projeto consolidou o uso de boas práticas de engenharia de software, em especial a separação em camadas (MVC) e o mapeamento objeto-relacional com herança..

Palavras chave: 3-5.

SUMÁRIO

1	Introdução	1
2	FERRAMENTAS E TECNOLOGIAS	1
3	DESENVOLVIMENTO	1
3.1	Levantamento dos Requisitos	1
3.2	Diagrama de Casos de Uso	1
3.3	Especificação dos Casos de Uso	1
3.4	Diagrama de Classes	1
3.5	Boas Práticas	2
3.6	Banco de Dados	2
3.7	Telas do sistema	2
3.8	Link dos códigos do projeto	2
4	CONCLUSÕES	2
	REFERÊNCIAS	3

1 INTRODUÇÃO

Plataformas como Amazon Web Services (AWS), Microsoft Azure e Google Cloud oferecem abstrações nas quais o usuário cria máquinas virtuais, anexa discos de armazenamento e acompanha o consumo de recursos por meio de painéis de monitoramento. Reproduzir, ainda que de forma simplificada, o funcionamento dessas plataformas é um exercício rico para a aplicação dos conceitos de orientação a objetos, pois o domínio é naturalmente hierárquico (diferentes tipos de recurso compartilham atributos e comportamentos comuns).

Para a construção do sistema empregaram-se métodos consolidados da engenharia de software: levantamento de requisitos funcionais e não funcionais, modelagem com diagramas UML(casos de uso e classes), arquitetura em camadas no padrão MVC e mapeamentoobjeto-relacional (ORM) com estratégia de herança. Os pilares da POO — herança,polimorfismo e encapsulamento — guiaram o projeto da classe abstrata Recurso e de suas especializações VirtualMachine e Armazenamento.

O objetivo deste projeto é desenvolver o WagaCloud, um simulador de plataforma de cloud computing que permita ao usuário autenticar-se, criar e gerenciar máquinas virtuais e discos de armazenamento, controlar o ciclo de vida desses recursos e acompanhar métricas simuladas de uso, demonstrando a aplicação prática dos conceitos de Programação Orientada a Objetos em um sistema completo e funcional, integrado a um banco de dados relacional.

2 FERRAMENTAS E TECNOLOGIAS

Para o desenvolvimento do WagaCloud foram utilizadas as seguintes ferramentas, linguagens e tecnologias:

Categoria	Ferramenta / Tecnologia
-----------	-------------------------

Linguagem de programação	Java 21 (LTS)
Framework de aplicação	Spring Boot 4.x (Spring Web MVC, Spring Data JPA)
Gerenciador de build e dependências	Maven (com <i>wrapper</i> <code>./mvnw</code>)
Sistema gerenciador de banco de dados (SGBD)	PostgreSQL
Mapeamento objeto-relacional (ORM)	Hibernate (via Spring Data JPA)
Redução de <i>boilerplate</i>	Lombok (@Getter, @Setter, @NoArgsConstructor)
Front-end	HTML5, CSS3 e JavaScript (API fetch), servidos estaticamente pelo Spring
IDE	Visual Studio Code (com extensões para Java/Spring)
Ferramenta CASE (modelagem UML)	Astah (arquivo docs/WagaClaude.astah)
Controle de versão	Git e GitHub

3 DESENVOLVIMENTO

A seguir são descritos os requisitos, diagramas e principais artefatos produzidos para o sistema, ilustrando as decisões de modelagem e implementação.

3.1 Levantamento dos Requisitos

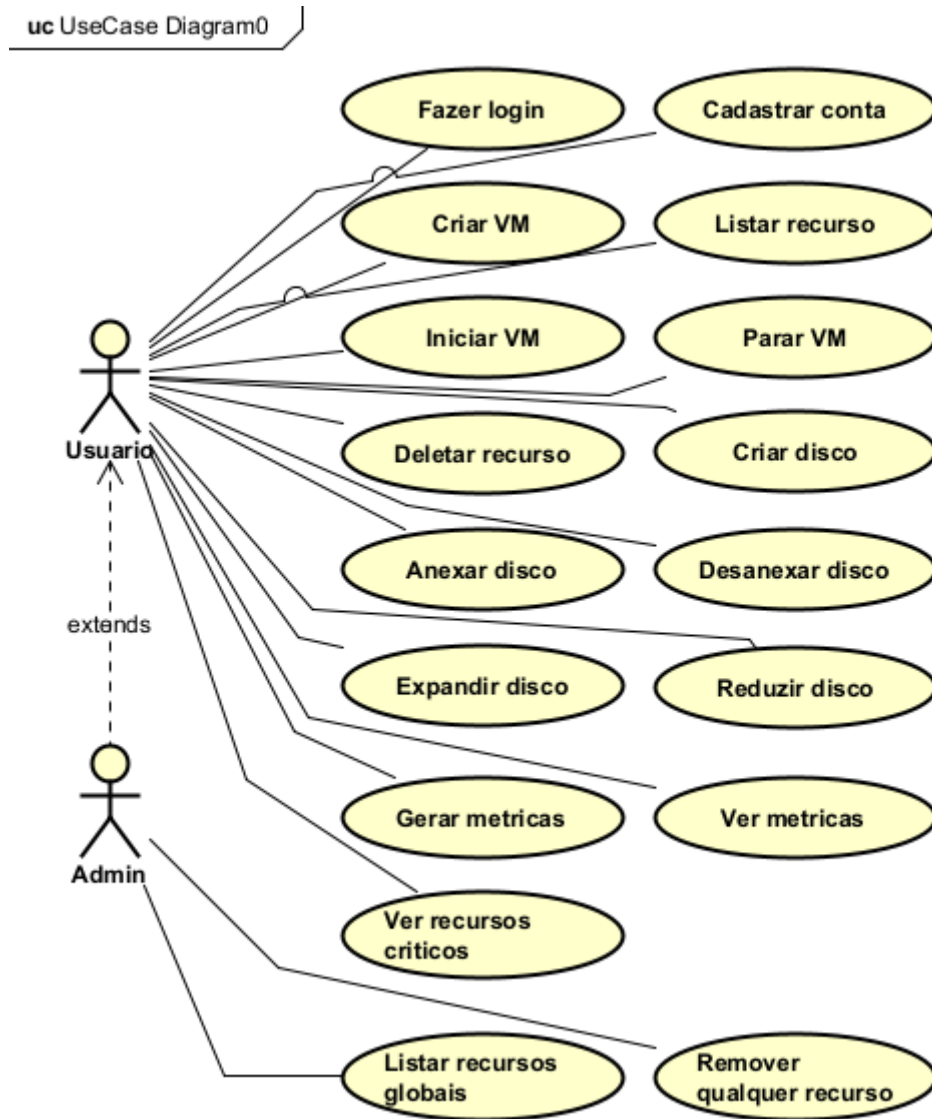
Requisitos Funcionais

Identificador	Descrição	Prioridade
RF01	O sistema deve possuir gerenciamento CRUD do próprio usuário.	Alta.
RF02	O usuário administrador deve conseguir gerenciar acesso e permissões de outros usuários.	Alta.
RF03	O sistema deve possuir dashboard que vai resumir as informações de suas instâncias/VMs.	Alta.
RF04	O usuário pode ter CRUD das instâncias/VMs da conta.	Alta.
RF05	O sistema deve ter uma aba para o gerenciamento de memória, podendo adicionar em alguma instância a qualquer momento.	Alta.
RF06	O sistema deve possuir autenticação de usuário.	Alta.

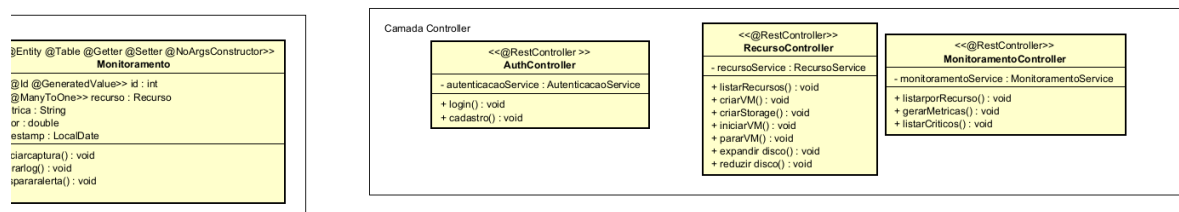
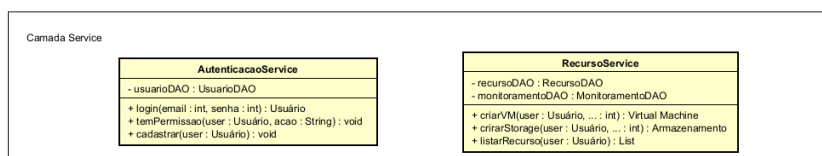
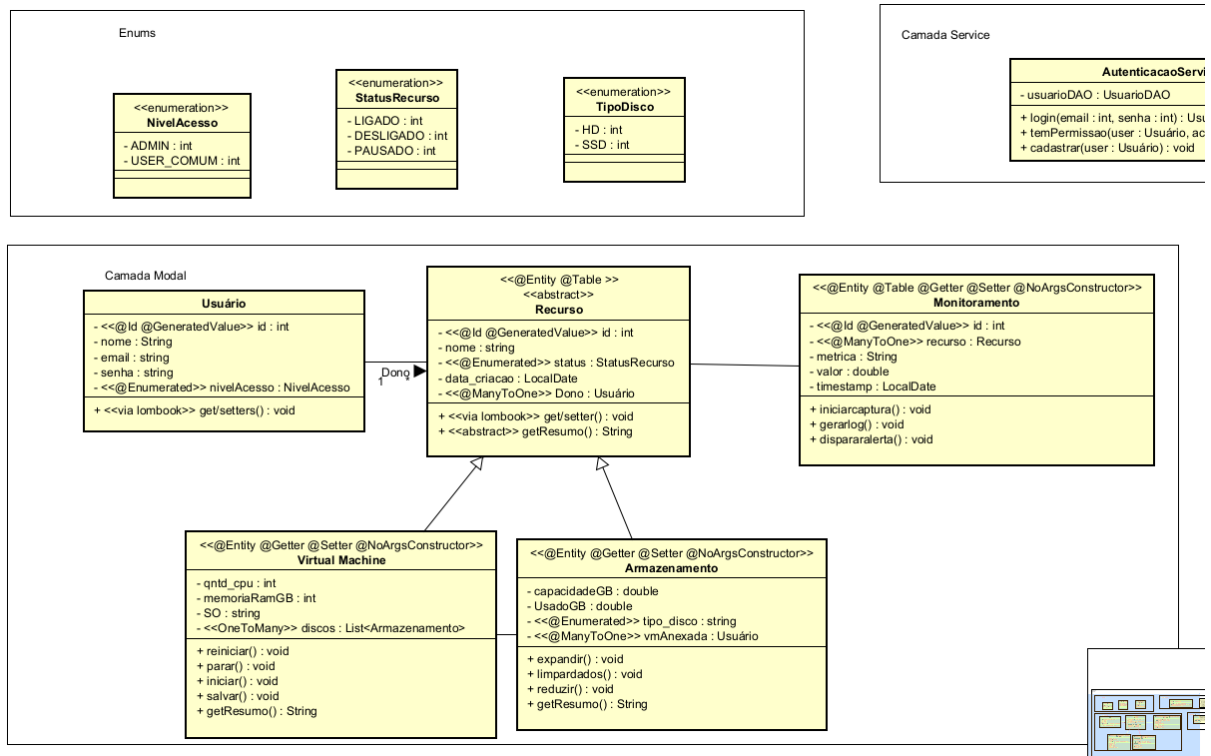
Requisitos não funcionais

Identificador	Descrição	Prioridade
RNF01	O sistema deve controlar a quantidade de recurso máximo a ser utilizada pelo contratante.	Eficiência/Espaço.
RNF02	As operações do sistema devem ser registradas em logs.	Segurança.
RNF03	A interface do sistema deve ser intuitiva, sendo separado em análise e gerenciamento.	Facilidade de uso.

3.2 Diagrama de Casos de Uso



3.3 Diagrama de Classes



3.4 Boas Práticas

Durante o desenvolvimento foram aplicadas, entre outras, as seguintes boas práticas:

Boa Prática 1 — Arquitetura em camadas (padrão MVC)

O projeto foi estruturado seguindo o padrão MVC, com separação rígida de

responsabilidades em camadas. A regra de ouro adotada é que nenhuma camada pula a

vizinha: o Controller nunca acessa o Repository diretamente — sempre passa pelo

Service, onde reside a lógica de negócio e a validação. A divisão dos arquivos em pacotes

evidencia a separação:

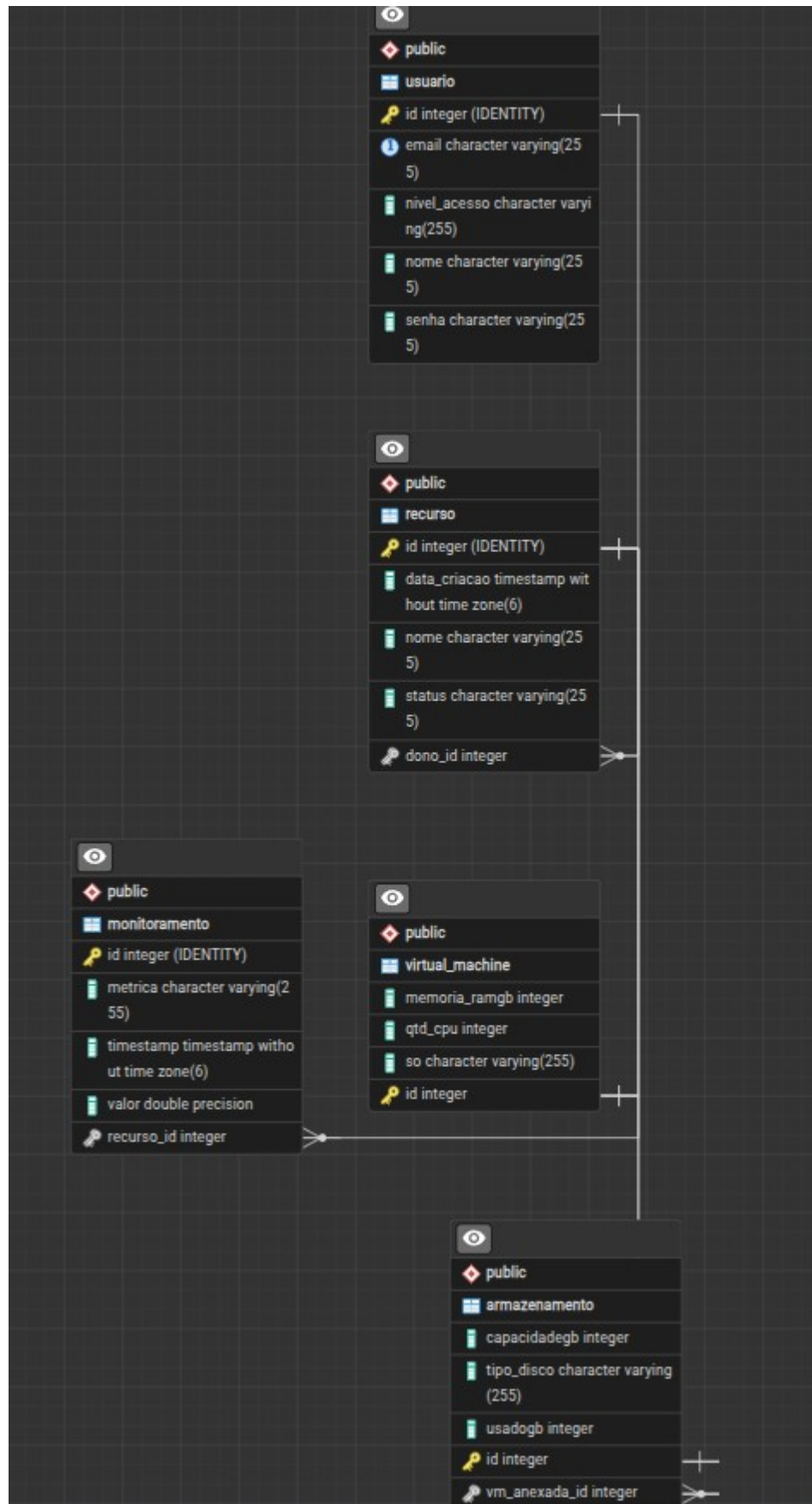
```
src/main/java/com/WagaCloud/wagacloud/
├─ controller/   ─ camada de apresentação (API REST: AuthController, RecursoController, MonitoramentoController)
├─ service/      ─ regras de negócio e validação (AutenticacaoService, RecursoService, MonitoramentoService)
├─ repository/   ─ acesso a dados (interfaces JpaRepository)
├─ model/        ─ entidades JPA (Recurso, VirtualMachine, Armazenamento, Usuario, Monitoramento)
├   └─ enums/    ─ NivelAcesso, StatusRecurso, TipoDisco
└─ config/       ─ DataLoader (carga inicial)
```

Boa Prática 2 — Encapsulamento e validação no domínio (regras junto da entidade)

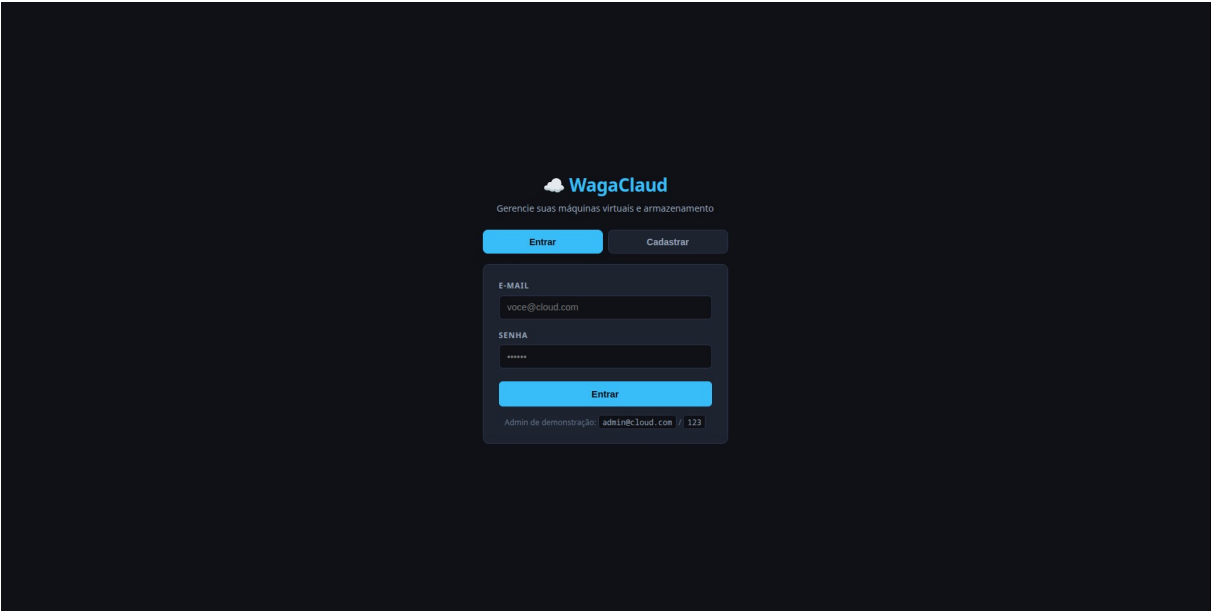
As regras que pertencem ao próprio objeto foram encapsuladas na entidade, em vez de espalhadas pelo sistema. A classe Armazenamento, por exemplo, protege seu estado: o método reduzir() impede que a capacidade fique abaixo do espaço já utilizado, garantindo um objeto sempre em estado consistente (encapsulamento de invariantes):

```
public void reduzir(int gb) {
    if (gb <= 0) {
        throw new IllegalArgumentException("A redução deve ser maior que zero");
    }
    if (this.capacidadeGB - gb < this.usadoGB) {
        throw new IllegalArgumentException(
            "Não é possível reduzir abaixo do espaço já usado (" + usadoGB + " GB)");
    }
    this.capacidadeGB -= gb;
}
```

3.5 Banco de Dados



3.6 Telas do sistema



WagaCloud

Recursos

Monitoramento

Administrador ADMIN Sair

NOVA MÁQUINA VIRTUAL

NOME

ex: web-app

VCPU

2

RAM (GB)

4

SISTEMA OPERACIONAL

ex: Ubuntu 22.04

Criar VM

Toda VM nova já vem com um disco padrão de 50 GB.

NOVO DISCO

NOME

ex: disco-extra

CAPACIDADE (GB)

100

USADO (GB)

0

TIPO

SSD

Criar disco

O disco nasce solto — depois você pode anexá-lo a uma VM.

MÁQUINAS VIRTUAIS

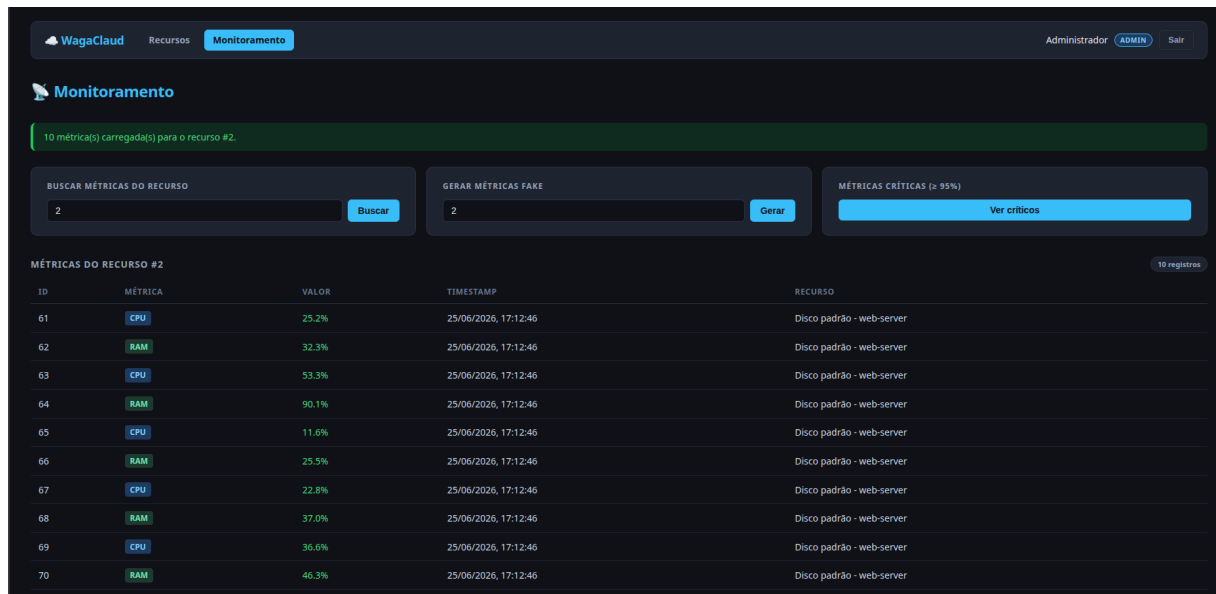
4 VMs

ID	NOME	STATUS	CPU	RAM	SO	DISCOS	DONO	AÇÕES
1	web-server	PARADO	4 vCPU	8 GB	Ubuntu 22.04	1 disco	Administrador	Iniciar Deletar
3	banco-dados	PARADO	8 vCPU	16 GB	Debian 12	1 disco	Maria Silva	Iniciar Deletar
5	app-backend	PARADO	2 vCPU	4 GB	Ubuntu 22.04	1 disco	João Souza	Iniciar Deletar
8	vm	ATIVO	2 vCPU	4 GB	Windows 11	2 discos	Lucas	Parar Deletar

DISCOS

6 discos - 1 solto

ID	NOME	STATUS	TIPO	USO	ANEXADO A	DONO	AÇÕES
2	Disco padrão - web-server	PARADO	SSD	0/50 GB	VM 'web-server'	Administrador	Expandir Reduzir Desanexar Deletar
4	Disco padrão - banco-dados	PARADO	SSD	0/50 GB	VM 'banco-dados'	Maria Silva	Expandir Reduzir Desanexar Deletar



3.7 Link do repositório

<https://github.com/LuBruck/Waga-Cloud>

4 CONCLUSÕES

O desenvolvimento do WagaCloud atingiu o objetivo proposto: foi construído um simulador de plataforma de cloud computing funcional, com autenticação, gerenciamento de máquinas virtuais e discos, e monitoramento por métricas simuladas, tudo persistido em banco relacional PostgreSQL. Os resultados obtidos confirmaram a adequação da modelagem orientada a objetos ao domínio: a classe abstrata Recurso e suas especializações permitiram tratar VMs e discos de forma uniforme (polimorfismo via `getResumo()`), reduzindo duplicação de código.